

CityMind; Phase 2 Final Report

Course: Artificial Intelligence **Program:** BS Computer Science **Group Members:** - 24i-0663; shared `CityGraph`, CSP (Ch1), Genetic Algorithm (Ch2), Simulated Annealing (Ch3), simulation runner + flood manager + risk shifts. - 24i-3033; A* router (Ch4), KMeans clustering (Ch5.1), synthetic data + Decision Tree classifier (Ch5.2), risk write-back to graph (Ch5.3), police deployment, Ursina UI collaboration (scene, HUD, setup pipeline, UI controls).

Phase 2 Deadline: 10th May. **Primary Language:** Python 3.

1. Executive Summary

CityMind is delivered as a fully integrated decision-support system. All five required challenges are implemented and operate on a single shared `CityGraph`. A 20-step simulation orchestrates floods, dynamic risk shifts, ambulance re-evaluation, and A*-based emergency replanning, with every component reading from and writing to the same graph object. A real-time 3D UI (Ursina) renders the city, all required overlay toggles, the ambulance team, the team mission, and a live event log.

This report documents what we built, where we deviated from our Phase-1 design document, why we deviated, and the work split per group member.

2. What was built (against rubric §2)

Challenge 1; City Layout Planning

- File: `citymind/layout_csp.py`.
- Algorithm: AC-3 to prune impossible values, Backtracking with MRV (minimum remaining values) and LCV (least constraining value) heuristics, plus a Min-Conflicts fallback for infeasible instances.
- Rule reporting: `LayoutResult.violation_breakdown` now reports per-rule counts (industrial-adjacency, residential 3-hop reach, power 2-hop reach, quota mismatch). The HUD top bar surfaces a compact summary like `Layout: 3 viol [Quota:2, IndAdj:1]` whenever the CSP cannot satisfy every rule.
- Live-modification: hop limits and the forbidden-neighbor set are constructor parameters (`hospital_max_hops`, `industrial_max_hops_for_power`, `industrial_forbidden_neighbors`). Changing the 3-hop rule to 2 hops is a single argument change and the system re-validates immediately.

Challenge 2; Road Network Optimization

- File: `citymind/road_ga.py`.
- Algorithm: Genetic Algorithm over an edge bitstring with tournament selection, uniform crossover, bitflip mutation, and an MST-seeded initial population (so half the individuals begin near a low-cost feasible region).
- Strict 2-edge-disjoint check (Menger's theorem, $k=2$): we replaced the prior path-then-ban heuristic with an Edmonds-Karp BFS-based max-flow on the capacity graph in which each undirected edge contributes two directed arcs of capacity 1. The fitness function uses the strict check, so the GA is rewarded only when there are genuinely two edge-disjoint paths between the primary hospital and the ambulance depot.
- Stagnation-based early stop with `patience=25` and `min_generations=30`, preserving determinism for any fixed seed.

Challenge 3; Ambulance Placement

- File: `citymind/ambulance_sa.py`.
- Algorithm: Simulated Annealing on the 3-tuple of grid cells with adaptive initial temperature (targeting ~80% acceptance of uphill moves), geometric cooling, and small-step neighbor moves with a low-probability random jump.
- Objective: minimax response distance over residential cells, computed via Dijkstra on the live `effective_cost` (so risk multipliers and blocked edges affect placements).
- Re-evaluation: triggered by the simulation runner when risk shifts pass a threshold during the tick loop.

Challenge 4; Emergency Routing

- Files: `citymind/astar_router.py`, `citymind/simulation.py`.
- Algorithm: A* with Manhattan $\times$ `min_edge_cost` heuristic. The heuristic is admissible because `min_edge_cost` is $\leq$ the smallest base cost on any edge. The router asserts this against `CityGraph.min_base_cost` at search time so future edits to the residential discount cannot silently break optimality.
- Mission: civilians are visited in order; the runner replans whenever the next planned edge becomes blocked.
- Repair fallback: if replanning fails for two consecutive ticks because all exits from the team are blocked, the runner restores the single blocked frontier edge that minimizes hops to the current target. This preserves forward progress in adversarial flood scenarios without requiring the team to teleport. Documented as a deliberate deviation in §3.

Challenge 5; Crime Risk Pipeline

- Files: `citymind/crime_kmeans.py`, `citymind/crime_risk_classifier.py`, `citymind/police_deployment.py`.
- Step 1 (unsupervised): K-Means with $k=3$ over engineered features (population density, Manhattan distance to nearest industrial, industrial cells within 3 hops, one-hot location

type). Cluster id is written back to the graph.

- Step 2 (synthetic data): $\text{rate} = w1 * \text{population} + w2 * \text{industrial_proximity} + w3 * \text{cluster_bias} + \text{Gaussian noise}$ (Default weights $w1=0.55, w2=0.30, w3=0.15$) are justified by the urban-crime intuition that population and proximity to industrial zones are leading indicators. Labels are assigned by terciles for a balanced training set.
- Step 3 (supervised): Decision Tree classifier ($\text{max_depth}=5, \text{min_samples_leaf}=2$). Accuracy and confusion matrix are reported. We chose Decision Tree over kNN because the splits are directly explainable in the viva.
- Risk write-back: predicted labels become `RiskLevel` on the graph; `risk_mult` (Low=1.0, Medium=1.5, High=2.0) propagates into `effective_cost` so Challenges 2, 3, and 4 all detour around High-risk zones automatically.
- Police deployment: 10 officers placed greedily to maximize total covered risk subject to a minimum-spacing constraint, satisfying the project statement's "10 police officers to deploy" requirement.

System integration

- File: `citymind/simulation.py`.
- Orchestration: `EmergencyMissionRunner` advances the simulation tick by tick. Each tick the `FloodEventManager` may block one edge (capped via `max_blocks_per_tick`); flooded edges optionally escalate the risk level of their endpoints (Low → Medium → High); a configurable threshold of risk shifts triggers SA re-evaluation; A* replans when the next planned edge is blocked.
- Single source of truth: every module reads/writes the same `CityGraph`. No module owns a private copy. Tests verify that risk and edge-block changes propagate to GA, SA, and A* costs.

User interface

- Files: `citymind/ui/app.py`, `citymind/ui/scene.py`, `citymind/ui/hud.py`, `citymind/ui/setup_pipeline.py`.
- Built with Ursina (3D). Window split into a top status bar, left controls sidebar, right inspector + legend + active overlays, bottom event log.
- Required overlay toggles: roads, ambulance coverage, risk heatmap.
- Bonus overlays: population (residential bar height + tint), KMeans cluster colors, GA hospital↔depot 2-edge-disjoint paths (color-coded path A, B, shared), A* replan trace, CSP diagnostic markers (red pillars on cells that violate hop rules), police pillars.
- Live event log with color coding, animated team movement (interpolation
- halo pulse), animated path "flow", ring on civilian rescue, modal summary panel at end of simulation, snapshot export.

3. Deviations from the Phase-1 Design Document

We deliberately deviated in five places. Each is justified below.

3.1 Pygame → Ursina (3D UI)

The design document specified a 2D Pygame UI with a tile grid plus a side panel of toggles. After implementing the core algorithms we re-evaluated the UI requirement. The project statement asks for overlay toggles, a real-time display, and a live event log; it does not constrain the rendering API.

We chose Ursina because:

- 3D rendering makes residential population (bar height) and risk heatmap (color tint) co-presentable at the same time without overlapping glyphs, which a 2D top-down grid struggles with.
- The mouse-driven `EditorCamera` allowed examiners to inspect the city from any angle during the demo, which we found dramatically improved comprehension of ambulance coverage and GA redundancy paths.
- Ursina is built on Panda3D and runs cross-platform, including on Windows where the team primarily develops.
- We did not need any feature unique to Pygame.

The algorithmic content is unchanged; only the rendering layer differs.

3.2 Repair fallback in the mission runner

The design document described replan-on-block but did not specify what to do if the team becomes fully isolated. In stress-test runs we observed that extreme flooding could leave the team with no exits at all, indefinitely. We added a deterministic single-edge "repair" mechanism: after `repair_after_failures` consecutive failed replans (default 2), the runner unblocks the single blocked edge on the team's reachable-set boundary that minimizes hops to the current target. This keeps the simulation progressing while still penalizing failures in the event log.

3.3 Strict edge-disjoint redundancy check

The design document described checking redundancy by finding a primary hospital→depot path and re-searching after banning each edge of that path. That heuristic is strictly weaker than 2-edge-connectivity (Menger's theorem). We replaced it with a max-flow-based check (Edmonds-Karp, capacity 1 per directed arc) that returns True iff there exist at least two edge-disjoint paths. This is provably correct and is what the project statement actually asks for.

3.4 Risk-driven SA re-evaluation threshold

The design said SA "re-evaluates as risk weights shift". We made this threshold-driven (`sa_reopt_threshold`, default 2 risk shifts within a tick) to avoid thrashing on every minor risk movement. The threshold is a constructor parameter and is exposed through `SetupPipeline`.

3.5 Parameterization for live-modification

The design document hard-coded the 3-hop hospital rule and the 2-hop industrial rule. To make the on-the-spot live-modification challenge trivial to demonstrate, we promoted these to constructor parameters (`hospital_max_hops` , `industrial_max_hops_for_power` , `industrial_forbidden_neighbors`) and propagated them through `SetupPipeline` . Changing a rule is now a one-line edit; the CSP re-validates immediately.

3.6 Additional overlays

The design listed three overlays (roads, coverage, risk heatmap). We added five more (population, clusters, GA redundancy paths, A* trace, CSP diagnostics) because each one directly visualizes an algorithm we implemented. They are all hidden by default and only the three required overlays are on at startup.

4. AI-Concept Coverage (rubric §3)

The rubric requires at least four distinct AI techniques. We use six:

1. **Constraint Satisfaction**; Ch1: AC-3, Backtracking (MRV/LCV), Min-Conflicts.
2. **Informed Search**; Ch4: A* with admissible heuristic.
3. **Evolutionary / Genetic Search**; Ch2: GA on edge bitstrings with MST seeding.
4. **Local Search**; Ch3: Simulated Annealing with adaptive initial temperature.
5. **Unsupervised Learning**; Ch5.1: K-Means.
6. **Supervised Learning**; Ch5.3: Decision Tree.

Auxiliary classical algorithms used but not counted as "AI techniques": Dijkstra (effective-cost shortest paths), BFS (hop distance and reachability), Edmonds-Karp max-flow (Menger's theorem for $k=2$), greedy maximum-coverage (police deployment).

5. Testing

The repository ships a `pytest` test suite covering:

- Shared graph: edges, risk multipliers, blocked state, accessibility.
- Challenge 1: AC-3 + backtracking determinism, infeasible-instance fallback, per-rule violation breakdown, quota-mismatch detection.
- Challenge 2: candidate-edge generation, connectivity penalty, the new strict 2-edge-disjoint check (cycle accepted, articulation rejected, bridge-only rejected), seed determinism.
- Challenge 3: SA state shape, objective improves over random baseline, determinism, dynamic cost awareness when risk and blocks change.
- Challenge 4: A* matches Dijkstra cost, finds alternates around blocks, reports unreachable, admissibility guard rejects too-high heuristic floor.

- Challenge 5: K-Means feature schema, deterministic clustering, synthetic label distribution, Decision Tree accuracy and confusion matrix shape, risk multiplier flow into all consumers.
- Police deployment: count cap, max-coverage objective beats random baseline, min-spacing enforced.
- Simulation runner: ordered target completion, replan on next-edge block, no revisit of completed targets, smoke logs, repair semantics, replan-fail log not spammed, risk shifts + SA re-eval triggered on flooded edges, direct unit tests of `_apply_risk_shifts`.
- Setup pipeline: byte-for-byte determinism, mission-seed sensitivity.
- UI smoke (skipped without display): single-tick run, every overlay toggle, cell click + inspector populated, snapshot export, summary modal.

Headless run:

```
pytest -q  
CITYMIND_SKIP_URSINA=1 pytest -q    # for CI without a display
```

6. Work Split

Component	Owner	Notes
<code>CityGraph</code> shared model	24i-0663	Foundation; co-reviewed before implementation
Challenge 1; CSP solver and infeasibility diagnostics	24i-0663	
Challenge 2; GA + MST seeding + strict redundancy check + early stop	24i-0663	
Challenge 3; Simulated Annealing	24i-0663	
Simulation runner + flood manager + risk shifts + repair fallback	24i-0663	
Challenge 4; A* router and admissibility guard	24i-3033	
Challenge 5.1; K-Means clustering	24i-3033	
Challenge 5.2; Synthetic data + Decision Tree	24i-3033	
Challenge 5.3; Risk write-back integration	24i-3033	
Police deployment (greedy max-coverage)	24i-3033	
Ursina 3D scene, HUD, setup pipeline, UI controls	Both	Co-designed and co-implemented; final UI behavior and controls validated jointly
Cross-module integration testing, snapshot/export, viva prep	Both	

7. Acceptance Criteria

- `pytest -q` passes locally on Windows.
- `CITYMIND_SKIP_URSINA=1 pytest -q` passes on a headless environment.
- `python app.py` exercises every challenge module in sequence.

- `python -m citymind.ui` launches the 3D UI; HUD shows layout status, required overlays toggle, live event log scrolls, snapshot export works.
 - Live-modification behavior is reproducible when seed values are recorded in HUD and event log.
-

End of Phase 2 Final Report.